SIGK - Zwiększanie rozdzielczości obrazu przy pomocy SRCNN oraz Cosine CNN

Jan Konarski - 347423

3 listopada 2026

1 Wstęp

Celem niniejszego zadania bylo sprawdzenie możliwości rekonstrukcji obrazu o niskiej rozdzielczości (32x32, oraz 64x64 piksele) do wysokiej (256x256 piksele). W projekcie wykorzystano zbiór DIV2K w celu przeprowadzenia treningu i walidacji skuteczności dwóch z zaimplementowanych sieci konwolucyjnych - jednej bazującej na klasycznej architekturze SRCNN, drugą oparto na filtrach asymetrycznych. Obie sieci korzystają z harmonogramu wyżarzania cosinusowego (CosineAnnealing). Aby zapewnić niezależność danych do testów zastosowano zbiór benchmarkowy BSD100. Do augmentacji danych treningowych zastosowano podstawowe operacje takie jak wycinanie losowego fragmentu obrazu (ang. cropping), odbicie lustrzane w dwóch wariantach (względem osi X lub Y), oraz losowy k-krotny obrót o 90 stopni.

2 Literatura

Niniejsze sieci powstały na bazie prac 'Image Super-Resolution Using Deep Convolutional Networks' - Chao Dong, Chen Change Loy, Kaiming He, et al.) oraz 'A Cosine Network for Image Super-Resolution' - Chunwei Tian, Chengyuan Zhang, Bob Zhang, et al.

3 Przygotowanie danych

Dane do zbioru danych DIV2K zostały pobrane z adresu `http://data.vision.ee.ethz.ch/cvl/DIV2K` natomiast zbiór BSD100 uzyskano przez API platformy Kaggle. W celu przygotowania wycinków o rozmiarze 256x256 pikseli do zbioru treningowego, stworzono klasę CropDataset oraz funkcję extract_crops, które dla ziarna generatora pseudolosowego wycinają losowy fragment obrazu wejściowego. Uzyskany w ten sposób obraz o wysokiej rozdzielczości poddawany jest operacjom augmentacji danych w celu zwiększenia różnorodności i zapobieganiu przeuczenia się sieci neuronowej. Następnie tworzony jest obraz o niskiej rozdzielczości, który uzyskiwany jest poprzez poddanie obrazu HR operacji pomniejszenia metodę cv2.resize. Jako interpolację wybrano metodę INTER_AREA, która jest zbliżona do metody najbliższych sąsiadów ze zredukowanym efektem mory - `https://docs.opencv.org/3.4.20/da/d54/group__imgproc__transform.html`

Dane są następnie przekształcane odpowiednio w zbiory - treningowy (6400 próbek - 8 wycinków na każde z 800 zdjęć), walidacyjny (100 próbek), oraz testowy (100 próbek). Po tym etapie rozpoczyna się trening sieci.

4 Architektura sieci

Sieć konwolucyjna SRCNN składa się z trzech warstw konwolucyjnych odpowiednio:

- Warstwa - Ekstrakcja Cech z obrazu RGB [1] - 3 kanały wejściowe, 64 filtry, jądro o rozmiarze K=9, oraz padding P=4.
- Warstwa - Mapowanie nieliniowe [2] - 64 kanały wejściowe, 32 filtry, jądro o rozmiarze K=1, oraz padding P=0.
- Warstwa - Rekonstrukcja do obrazu RGB [3] - 32 kanały wejściowe, 3 kanały wyjściowe, jądro o rozmiarze K=5, padding P=2.

Parametry te zostały dobrane na bazie literatury, umożliwiając efektywny trening sieci splotowej. Funkcją aktywacji sieci jest ReLU (stosowana dla warstwy 1 oraz 2), zastosowane zostały również globalne połączenia rezydualne umożliwiające uczenie się od obrazu interpolowanego bikubicznie.

Zastosowaną w pracy funkcją straty jest funkcja Charbonniera definiowana jako:

$$Loss = \frac{1}{N} \sum_{i=1}^{N} \sqrt{(\hat{y}_i - y_i)^2 + \epsilon^2}$$

Gdzie $\hat{y}$ jest predykcją (rekonstrukcją) obrazu przez sieć neuronową, y jest obrazem o wysokiej rozdzielczości a parametr ϵ wprowadza stabilność numeryczną. Powyższą funkcję wybrano celowo, wcześniejsze etapy projektowe ujawniły, że zastosowanie MSE przekłada się na wyższe rozmycie rekonstruowanego obrazu (kwadratowa kara za duże błędy).

```python
class CharbonnierLoss(nn.Module): # Funkcja straty Charbonniera, podobna do L1 ale zapewnia większą stabilność nu
  def __init__(self, eps=EPS):    # Dla zadania SR strata MSE generuje wyblurrowany obraz, nie jest to najlepszy
                                   # W artykule dla CosineSRCNN stosowane jest zwykłe MAE (L1).
    super(CharbonnierLoss, self).__init__()
    self.eps = eps

  def forward(self, prediction, target):
    diff = prediction - target
    loss = torch.mean(torch.sqrt(diff * diff + self.eps * self.eps))
    return loss
```

Rysunek 1: Implementacja funkcji straty.

Sieć CosineSRCNN została zbudowana na wzór architektury CSRNet w uproszczonej formie. Do jej implementacji utworzono dwa bloki - OEB (Odd Enhancement Block) oraz EEB (Even Enhancement Block). OEB sumuje splot wzmocnionych cech asymetrycznych (równolegle 3x3, 1x3, 3x1) wraz z połączeniem rezydualnym po czym stosuje funkcję ReLU. EEB łącząc dwa sploty kernelem o rozmiarze K=3 wprowadza połączenia rezydualne. Sieć składa się ze splotu wejściowego 3x3, czterech bloków ułożonych naprzemiennie (OEB, EEB, OEB, EEB) i splotu wyjściowego 3x3 rekonstruując obraz RGB.

Podczas gdy pierwsza z sieci ma około 20 tyś. parametrów $(3 \cdot 64 \cdot 81 + 64) + (64 \cdot 32 \cdot 1 + 32) + (32 \cdot 3 \cdot 25 + 3) = 20099$.

Druga z sieci posiada około 270 tyś. parametrów, umożliwiając jej na głębszy proces uczenia.

Tabela 1: Porównanie sieci pod względem rozmiaru i kosztu obliczeniowego.

Sieć	Parametry	Czas / epoka	Uwagi
SRCNN	20 099	ok. 30 s	ostatnia warstwa zerowana, resid. globalny
CosineSRCNN	ok. 274 500	ok. 6 min	64 kanały, OEB/EEB × 2, resid. globalny

```python
class SRCNN(nn.Module):
  def __init__(self):
    super().__init__()
    self.feature_extraction = nn.Sequential(
      # Na początku mamy 3 kanały wejściowe RGB, stosujemy 64 filtry o rozmiarze kernela 9x9.
      nn.Conv2d(in_channels=3, out_channels=64, kernel_size=9, padding=4),
      nn.ReLU(),
      nn.Conv2d(in_channels=64, out_channels=32, kernel_size=1, padding=0),
      nn.ReLU(),
      nn.Conv2d(in_channels=32, out_channels=3, kernel_size=5, padding=2) # Dajemy na wyjście RGB - dobór paddin
      # ale idea jest mniej więcej taka, że obraz wejściowy jest rozdzielczości M x M i na wyjściu mamy tą samą ro
    )
    nn.init.zeros_(self.feature_extraction[-1].weight)
    nn.init.zeros_(self.feature_extraction[-1].bias) # Początkowa inicjalizacja zerami dla biasów i wag.
    # Chcemy na początku treningu przekazać do sieci niezmodyfikowany obrazek, dzięki temu funkcja straty nie robi

  def forward(self,x):
    return x + self.feature_extraction(x)

  @torch.no_grad()
  def predict(self, x):
    return torch.clamp(self.forward(x), 0.0, 1.0)
```

Rysunek 2: Implementacja sieci splotowej SRCNN.

```python
class OEB(nn.Module):
  def __init__(self, channels):
    super().__init__()
    self.conv3x3 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
    self.conv1x3 = nn.Conv2d(channels, channels, kernel_size=(1,3), padding=(0,1))
    self.conv3x1 = nn.Conv2d(channels, channels, kernel_size=(3,1), padding=(1,0))
    self.activation = nn.ReLU()
  def forward(self,x):
    out1 = self.conv3x3(x)
    out2 = self.conv1x3(x)
    out3 = self.conv3x1(x)
    sum_out = out1 + out2 + out3 + x
    return self.activation(sum_out)
```

```python
class EEB(nn.Module):
  def __init__(self, channels):
    super().__init__()
    self.conv_block = nn.Sequential(
        nn.Conv2d(channels, channels, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.Conv2d(channels, channels, kernel_size=3, padding=1)
    )
    self.activation = nn.ReLU()
  def forward(self,x):
    out = self.conv_block(x)
    return self.activation(out+x)
```

Rysunek 3: Odd Enhancement Block oraz Even Enhancement Block inspirowane siecią CSRNet.

```python
class CosineSRCNN(nn.Module):
  def __init__(self, channels=64):
    super().__init__()
    # Przechodzimy z 3 kanałów wejściowych (RGB)
    self.head = nn.Conv2d(3, channels, kernel_size=3, padding=1)
    self.body = nn.Sequential(
        OEB(channels),
        EEB(channels),
        OEB(channels),
        EEB(channels)
    )
    # Rekonstrukcja obrazu po ekstrakcji cech asymetrycznych (OEB) i stabilizacji (EEB)
    self.tail = nn.Conv2d(channels, 3, kernel_size = 3, padding=1)
  def forward(self,x):
    features = self.head(x)
    detail_features = self.body(features)
    return self.tail(detail_features) + x
```

Rysunek 4: Implementacja sieci splotowej Cosine SRCNN.

```python
train_losses = []
val_losses = []

best_val_loss = float('inf')

optimizer = optim.Adam(model.parameters(), lr=LR, betas=(0.9, 0.999), eps=1e-8) # Klasyczny Adam jako optimizer.
scheduler = CosineAnnealingLR(optimizer, T_max=NUM_EPOCHS, eta_min=LR_MIN) # Automatyczne 'ścinanie' learning ra

print("Started training model...")
for epoch in range(NUM_EPOCHS):
  train_loss = run_epoch(model, train_loader, criterion, optimizer)
  scheduler.step()
  val_loss = run_epoch(model, val_loader, criterion)
  train_losses.append(train_loss)
  val_losses.append(val_loss)
  print(f"Epoch {epoch+1}/{NUM_EPOCHS} | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}"
        f" (Bicubic: {baseline_val_loss:.6f}) | LR: {scheduler.get_last_lr()[0]:.1e}")
  if val_loss < best_val_loss:
    best_val_loss = val_loss
    torch.save({'epoch': epoch + 1, 'model_state_dict': model.state_dict(),
      'optimizer_state_dict': optimizer.state_dict(),
      'train_loss': train_loss, 'val_loss': val_loss}, f'{model_name}_x{UPSCALE_FACTOR}_best.pth') # Zapisywanie
```

Rysunek 5: Pętla treningowa sieci neuronowej.

5 Przebieg eksperymentów

W trakcie budowy sieci neuronowej napotkano kilka problemów, pierwszy z nich wynikał z zastosowania dynamicznego podziału danych (slider w Colab) wraz z RandomCrop i CenterCrop. Wprowadzenie tego mechanizmu powodowało, że część danych w zbiorze treningowym posiadała zasadniczy bias (tylko środkowy wycinek). Ponadto uzyskiwany w ten sposób zbiór danych był bardzo mały. Rozwiązano to stosując kilkukrotne wycinanie losowe dla całego zbioru treningowego, dzięki czemu uzyskano znacznie większy zbiór do treningu sieci. Po wprowadzeniu tej zmiany wyniki sieci przewyższyły metodę bazową (wcześniej około 1 dB gorsza od bicubic baseline). Inną zmianą było ustandaryzowanie przestrzeni barwnej na etapie testowania i walidacji, w tym celu dokonano zmiany na przestrzeń YCrCb. Kolejnym problemem okazała się moc przerobowa środowiska wykonawczego i wąskie gardło w trakcie pracy SRCNN. Wcześniejsze pomiary czasu pracy siatki wskazywały na to, że trening jednej epoki SRCNN wymagał około 2 minut. Zastosowanie wielu procesów w `DataLoader` (`num_workers=2`) oraz `nn.DataParallel` skróciło czas epoki z ok. 2 minut do ok. 30 sekund. Ponieważ obiekt `DataParallel`

na dwóch GPU może dać co najwyżej dwukrotne przyspieszenie, znaczną część zysku należy przypisać wyeliminowaniu wąskiego gardła w ładowaniu i augmentacji danych (dane wycinków są przechowywane w pamięci RAM środowiska).

5.1 Wyniki treningu

Podczas gdy pierwsze z wytrenowanych sieci osiągały niskie wyniki w ramach metryk PSNR, SSIM, LPIPS - kolejne treningi pozwoliły udokumentować poprawę jakości obrazu rekonstruowanego względem interpolacji bikubicznej.

Tabela 2: Porównania parami przy tej samej skali. Różnica = (pierwszy model) − (drugi model).

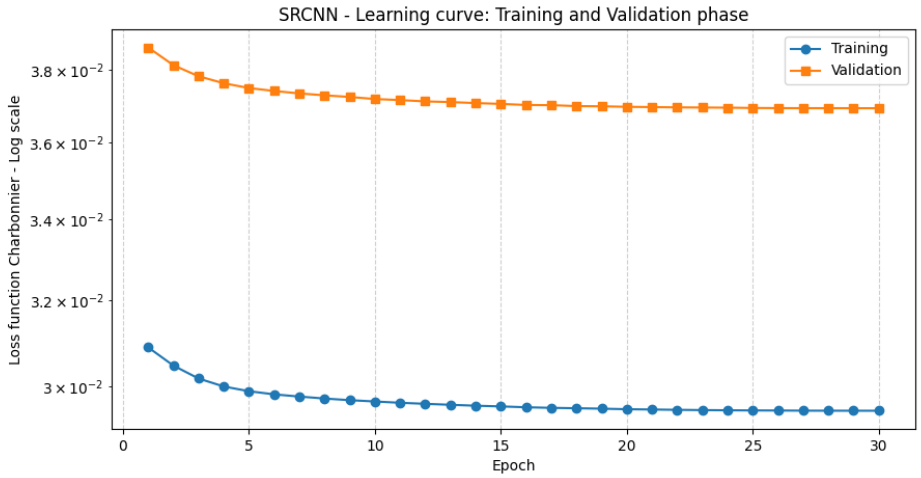
Porównanie	ΔPSNR [dB]	ΔSSIM	ΔLPIPS
x4: SRCNN 60 ep. vs SRCNN 30 ep.	+0.086	+0.0037	−0.020
x8: SRCNN 60 ep. vs SRCNN 30 ep.	+0.014	+0.0019	−0.022
x4: CosineSRCNN 30 ep. vs SRCNN 60 ep.	+0.158	+0.0076	−0.022
x8: CosineSRCNN 5 ep. vs SRCNN 30 ep.	−0.073	−0.0048	+0.033
x8: CosineSRCNN 5 ep. vs SRCNN 60 ep.	−0.087	−0.0067	+0.055

5.1.1 Trening SRCNN 1, Upscaling 4x

Parametry zadania: 30 Epok, Obraz low-resolution 64x64 pikseli rozdzielczości, 32 próbki w batchu sieci.

Tabela 3: Wyniki ewaluacji na zbiorze BSD100 (obrazy HR 256×256). Pogrubiono najlepszy wynik w danej skali.

Skala	Model	Epoki	Batch	PSNR [dB] ↑	SSIM ↑	LPIPS ↓
x4 (LR 64×64)						
	Bicubic	—	—	24.0230	0.6607	0.5192
	SRCNN	30	32	24.4397	0.6870	0.4472
	SRCNN	60	16	24.5262	0.6907	0.4268
	CosineSRCNN	30	16	**24.6844**	**0.6983**	**0.4043**
x8 (LR 32×32)						
	Bicubic	—	—	21.7059	0.4914	0.7207
	SRCNN	30	16	21.9765	0.5111	0.6547
	SRCNN	60	16	**21.9907**	**0.5130**	**0.6327**
	CosineSRCNN	5	16	21.9035	0.5063	0.6876



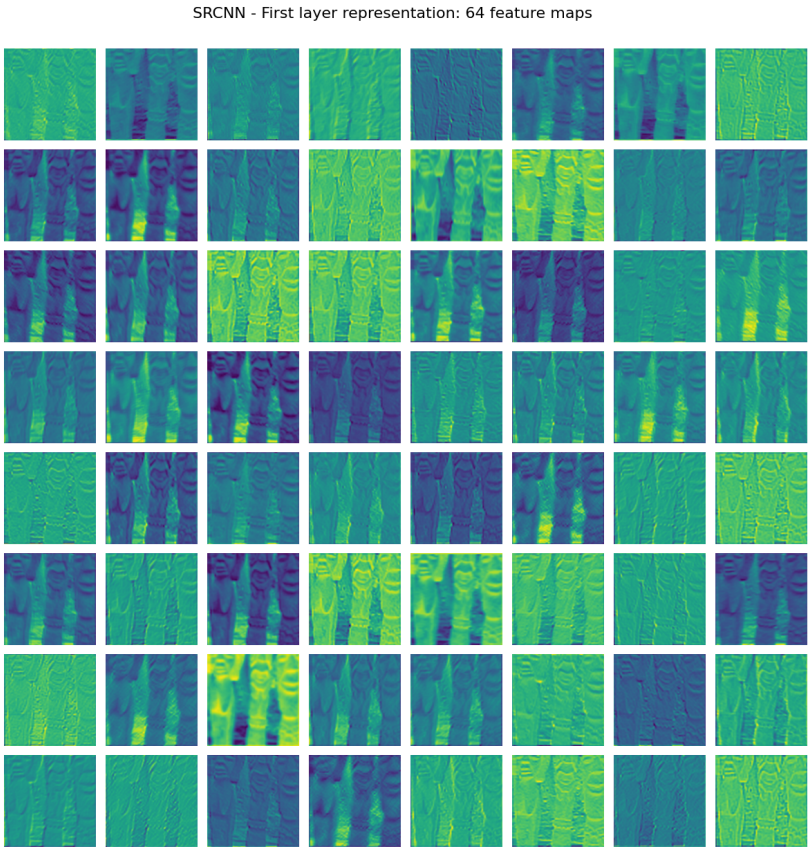
Rysunek 6: Wyniki treningu sieci dla LR=64x64, epochs=30, batch_size=32.



Rysunek 7: Rekonstrukcja obrazu przez sieć splotową SRCNN.

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 24.439719 dB | SSIM: 0.687038 | LPIPS: 0.447188
Bicubic Upscale | PSNR: 24.022980 dB | SSIM: 0.660729 | LPIPS: 0.519213
```

Rysunek 8: Wyniki ewaluacji PSNR, SSIM, LPIPS.



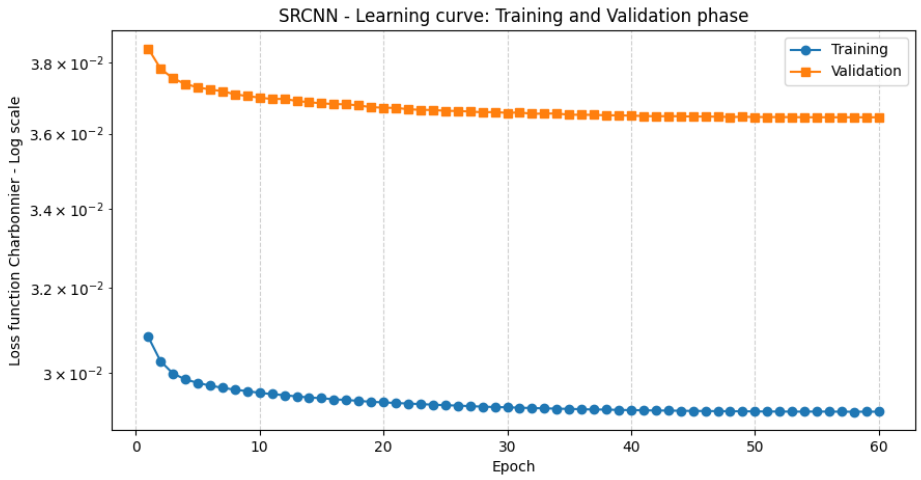
Rysunek 9: Mapa aktywacji filtrów pierwszej warstwy SRCNN.

Obserwacje: Rozmiar batcha zmieniono z 32 na 16 w kolejnych eksperymentach, nie sprawdzano jego dokładnego wpływu na przebieg uczenia. W pozostałych eksperymentach zadecydowano o pozostaniu przy 16. Trening zajął około 15 minut. Na wytrenowanym modelu odnotowano lekki wzrost PSNR (+0.4167) oraz SSIM (+0.0263), a także oczekiwany spadek LPIPS do 0.4471. Wizualnie uzyskany wynik jest niewiele lepszy od metody bikubicznej, ale uchwytuje więcej krawędzi i drobnych detali traconych przez interpola-

cję bikubiczną. Zaobserwowane spłaszczenie krzywych uczenia jest częściowo skutkiem zastosowania harmonogramu uczenia CosineAnnealing (wyżarzanie cosinusowe).

5.1.2 Trening SRCNN 2, Upscaling 4x

Parametry zadania: 60 Epok, Obraz low-resolution 64x64 pikseli rozdzielczości, 16 próbki w batchu sieci.



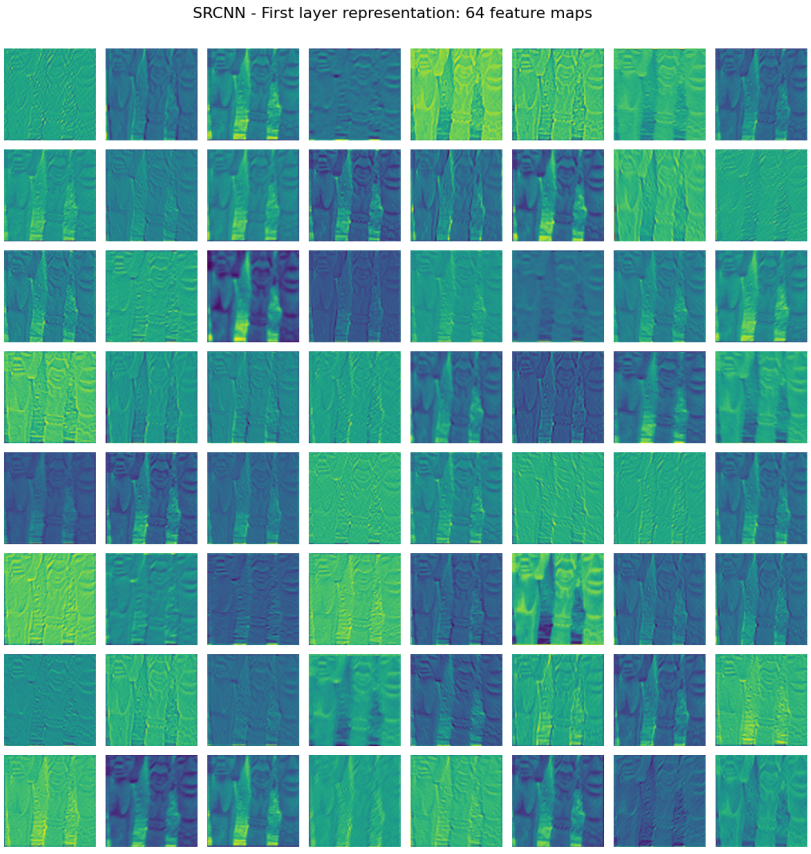
Rysunek 10: Wyniki treningu sieci dla LR=64x64, epochs=60, batch_size=16.



Rysunek 11: Rekonstrukcja obrazu przez sieć splotową SRCNN.

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 24.526180 dB | SSIM: 0.690713 | LPIPS: 0.426757
Bicubic Upscale | PSNR: 24.022980 dB | SSIM: 0.660729 | LPIPS: 0.519213
```

Rysunek 12: Wyniki ewaluacji PSNR, SSIM, LPIPS.

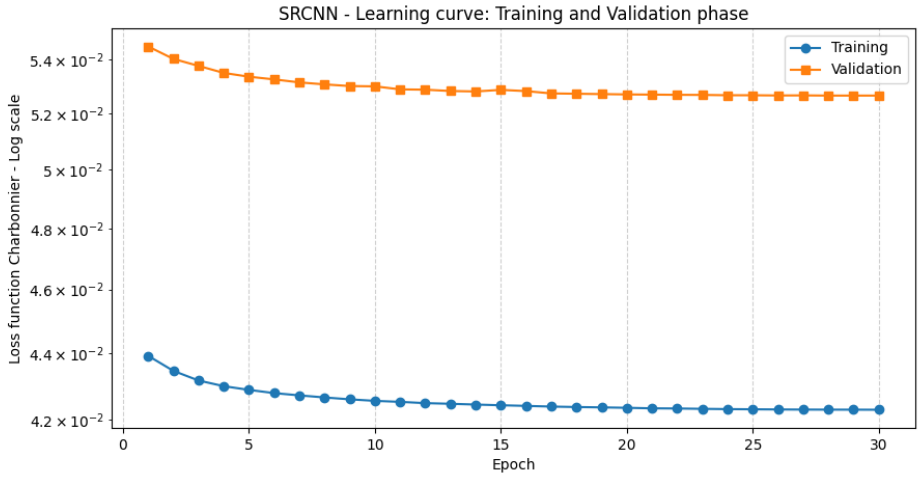


Rysunek 13: Mapa aktywacji filtrów pierwszej warstwy SRCNN.

Obserwacje: Trening 60 epok trwał dwukrotnie dłużej niż poprzedni. Względem metody bazowej uzyskano wzrost PSNR o 0.503 dB, SSIM o 0.030 oraz spadek LPIPS o 0.092. W porównaniu z treningiem 30-epokowym przyrost jest niewielki: +0.086 dB PSNR, +0.004 SSIM i -0.020 LPIPS.

5.1.3 Trening SRCNN 3, Upscaling 8x

Parametry zadania: 30 Epok, Obraz low-resolution 32x32 pikseli rozdzielczości, 16 próbki w batchu sieci.



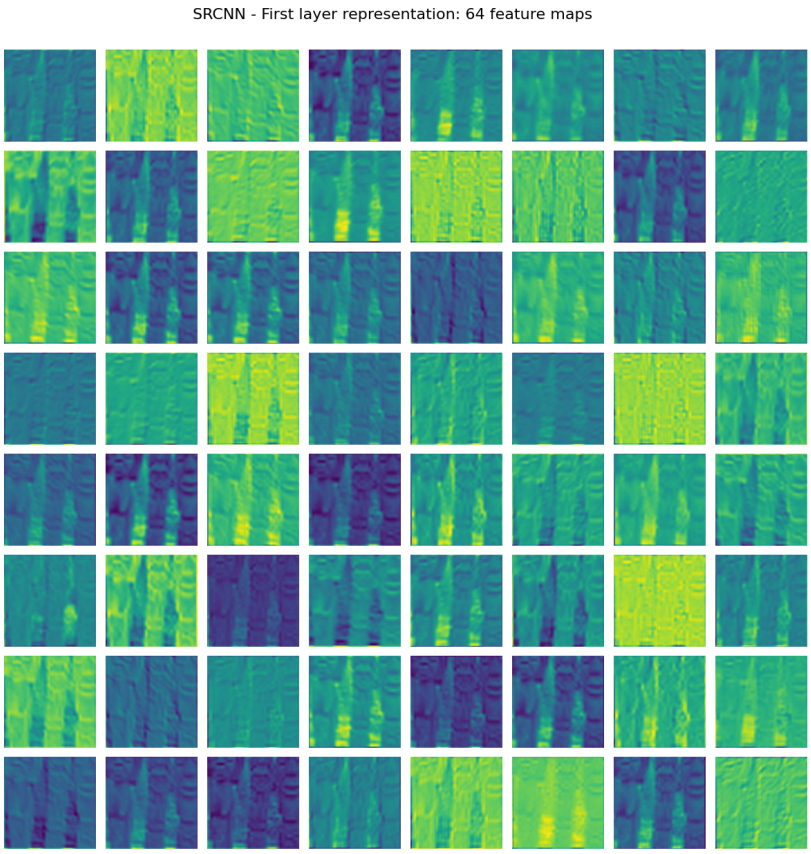
Rysunek 14: Wyniki treningu sieci dla LR=32x32, epochs=30, batch_size=16.



Rysunek 15: Rekonstrukcja obrazu przez sieć splotową SRCNN.

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 21.976544 dB | SSIM: 0.511071 | LPIPS: 0.654652
Bicubic Upscale | PSNR: 21.705900 dB | SSIM: 0.491392 | LPIPS: 0.720687
```

Rysunek 16: Wyniki ewaluacji PSNR, SSIM, LPIPS.

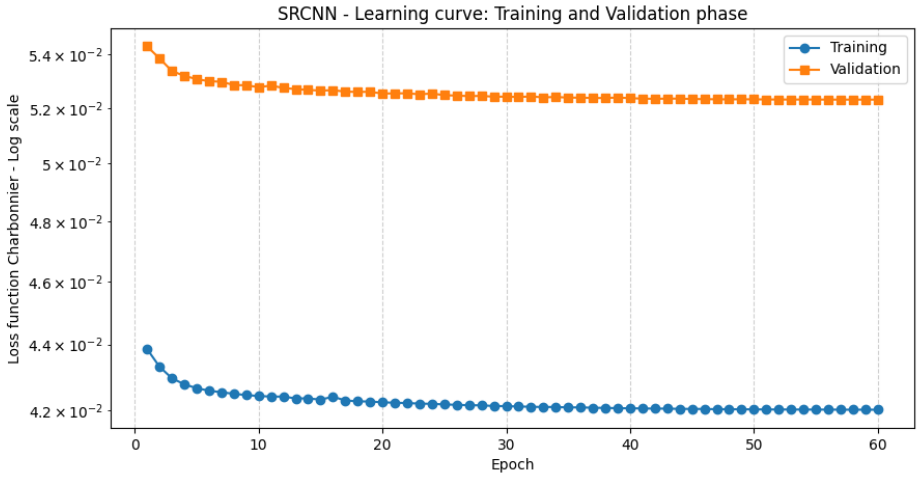


Rysunek 17: Mapa aktywacji filtrów pierwszej warstwy SRCNN.

Obserwacje: Trening 30-epokowy dla skali 8x przyniósł wzrost PSNR o 0.271 dB, SSIM o 0.020 oraz spadek LPIPS o 0.066 względem metody bazowej. Dla porównania przy tej samej liczbie epok i skali 4x zyski wyniosły odpowiednio +0.417 dB, +0.026 i -0.072.

5.1.4 Trening SRCNN 4, Upscaling 8x

Parametry zadania: 60 Epok, Obraz low-resolution 32x32 pikseli rozdzielczości, 16 próbki w batchu sieci.



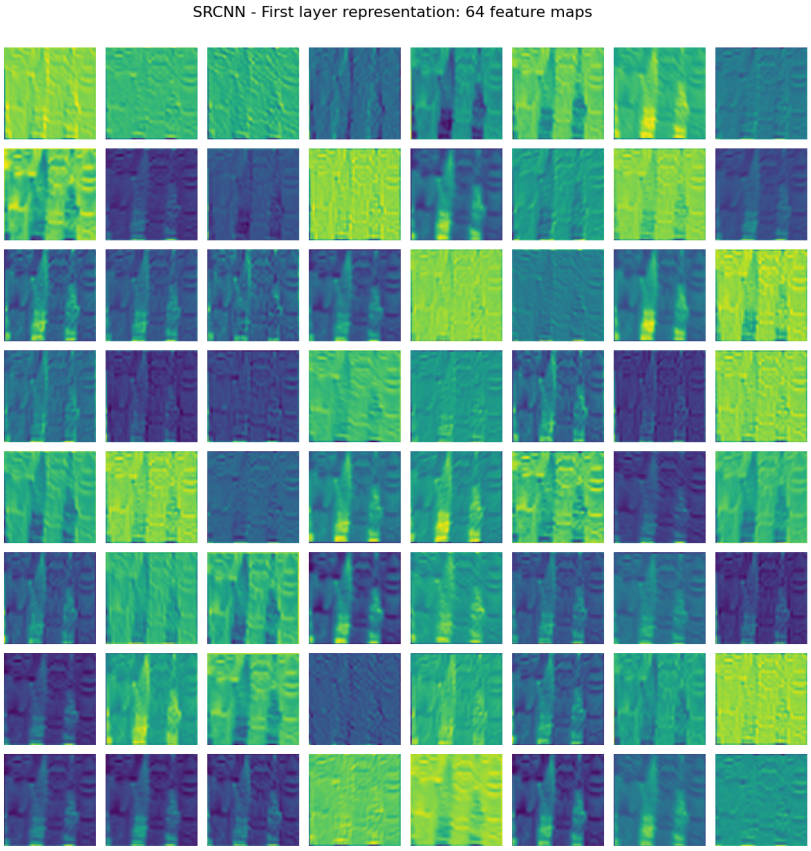
Rysunek 18: Wyniki treningu sieci dla LR=32x32, epochs=60, batch_size=16.



Rysunek 19: Rekonstrukcja obrazu przez sieć splotową SRCNN.

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 21.990660 dB | SSIM: 0.512997 | LPIPS: 0.632723
Bicubic Upscale | PSNR: 21.705900 dB | SSIM: 0.491392 | LPIPS: 0.720687
```

Rysunek 20: Wyniki ewaluacji PSNR, SSIM, LPIPS.

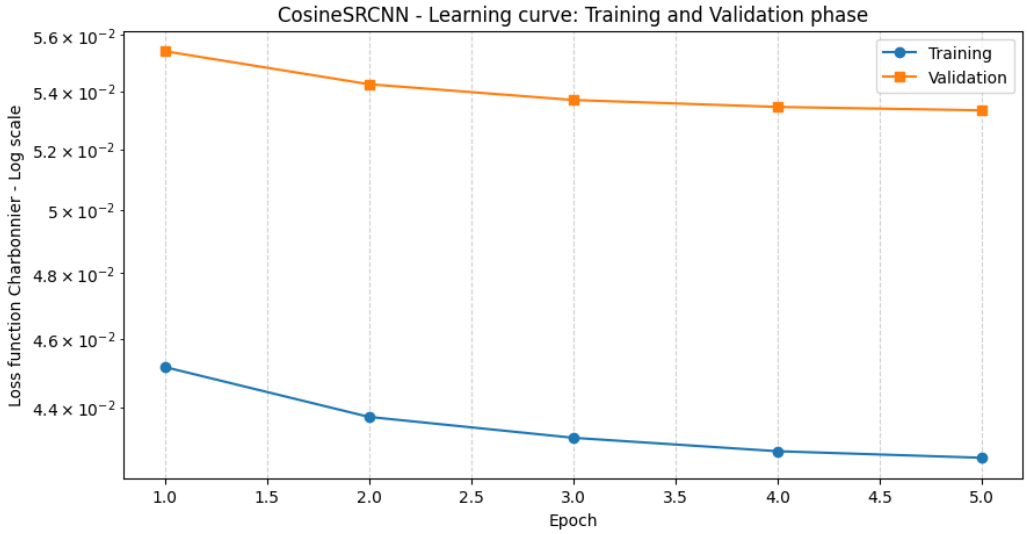


Rysunek 21: Mapa aktywacji filtrów pierwszej warstwy SRCNN.

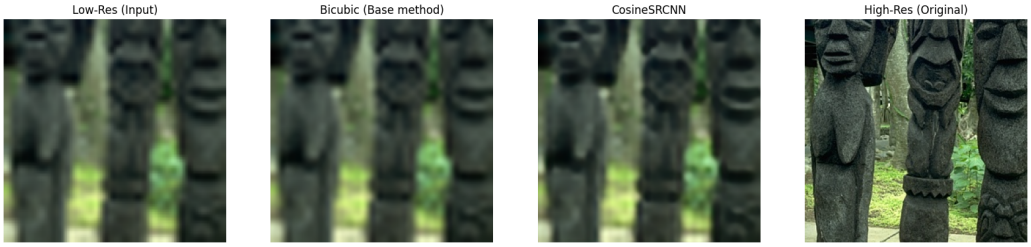
Obserwacje: Wydłużenie treningu z 30 do 60 epok dla skali 8x dało jedynie niewielką poprawę: PSNR wzrósł o 0.014 dB, SSIM o 0.002, a LPIPS spadł o 0.022. Spadek LPIPS jest zbliżony do tego, jaki zaobserwowano dla skali 4x (0.020). Różnice te są małe i, wobec braku powtórzeń eksperymentu z różnymi ziarnami, nie da się stwierdzić, czy są istotne statystycznie. Wizualnie rekonstrukcja nadal jest wyraźnie rozmyta dla odbiorcy.

5.1.5 Trening Cosine SRCNN 1, Upscaling 8x

Parametry zadania: 5 Epok, Obraz low-resolution 32x32 pikseli rozdzielczości, 16 próbki w batchu sieci.



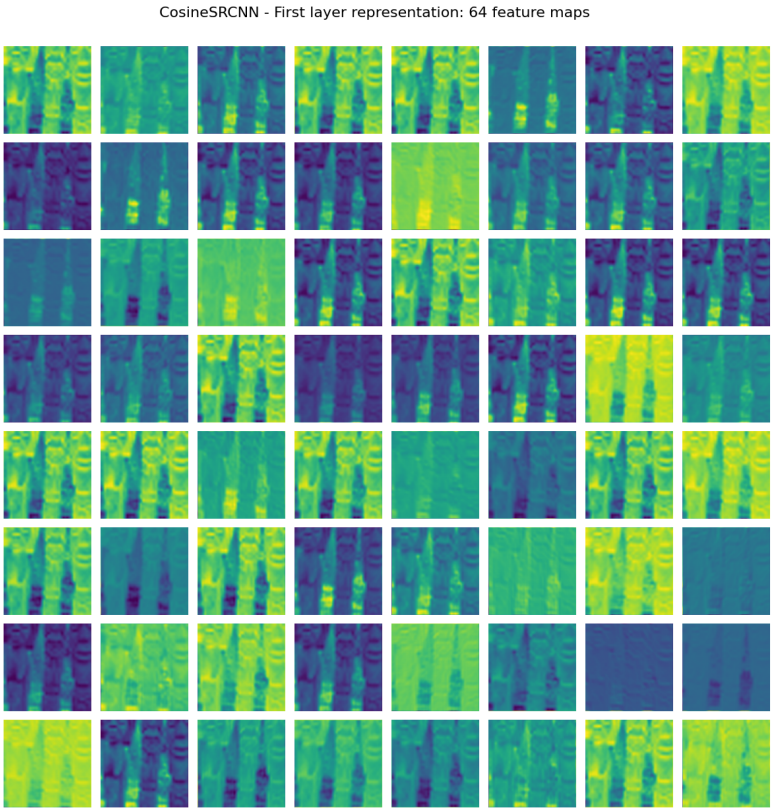
Rysunek 22: Wyniki treningu sieci dla LR=32x32, epochs=5, batch_size=16.



Rysunek 23: Rekonstrukcja obrazu przez sieć Cosine SRCNN.

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 21.903462 dB | SSIM: 0.506320 | LPIPS: 0.687618
Bicubic Upscale | PSNR: 21.705900 dB | SSIM: 0.491392 | LPIPS: 0.720687
```

Rysunek 24: Wyniki ewaluacji PSNR, SSIM, LPIPS dla CosineSRCNN po 5 epokach (x8)

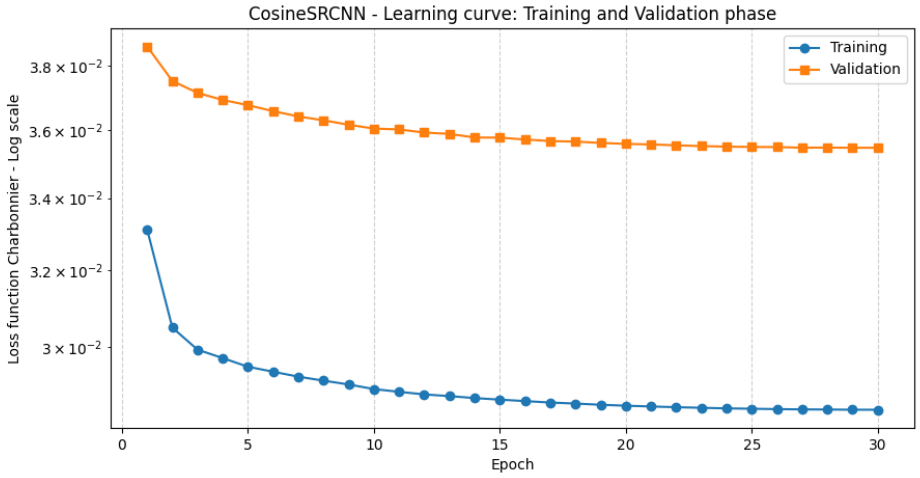


Rysunek 25: Mapa aktywacji filtrów pierwszej warstwy Cosine SRCNN.

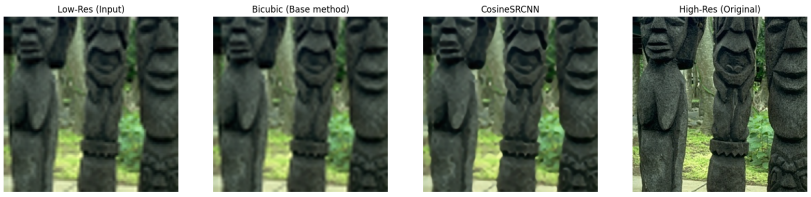
Obserwacje: Trening 5 epok sieci CosineSRCNN trwał ok. 30 minut, czyli tyle samo co 60 epok sieci SRCNN. Uzyskane metryki (PSNR 21.903 dB, SSIM 0.506, LPIPS 0.688) są nieznacznie gorsze od SRCNN trenowanej przez 30 epok. Wszystkie trzy wyraźnie przewyższają interpolację bikubiczną. Widoczna jest różnica w LPIPS względem SRCNN dla 60 epok (0.688 wobec 0.633).

5.1.6 Trening Cosine SRCNN 2, Upscaling 4x

Parametry zadania: 30 Epok, Obraz low-resolution 64x64 pikseli rozdzielczości, 16 próbki w batchu sieci.



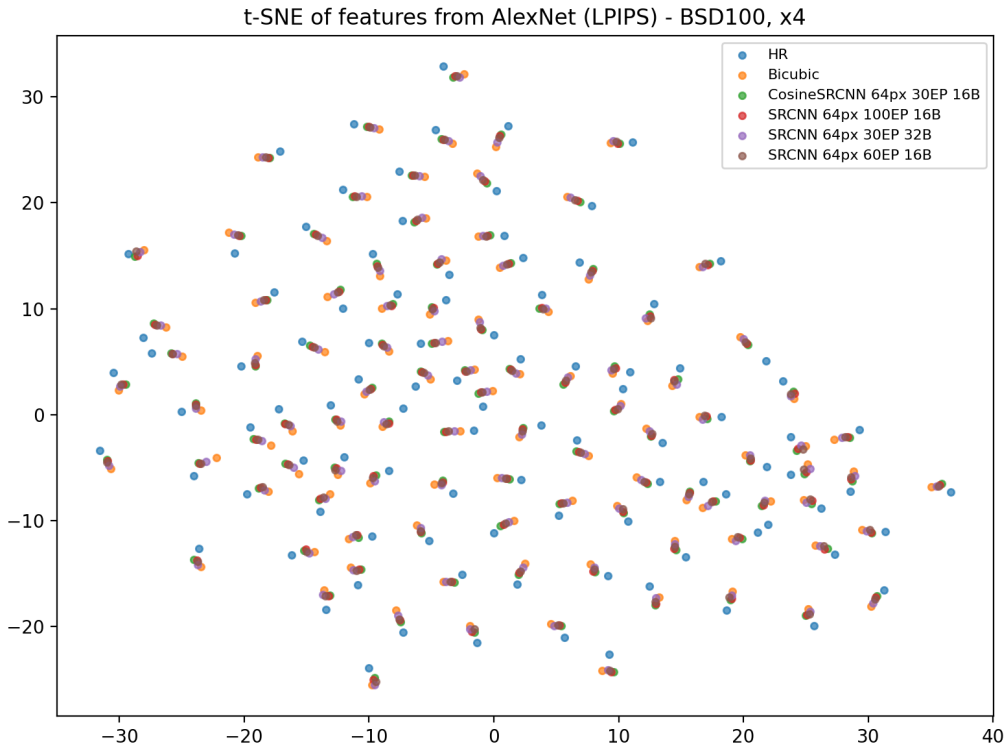
Rysunek 26: Wyniki treningu sieci dla LR=64x64, epochs=30, batch_size=16. Warto zauważyć gwaltowny spadek funkcji kosztu w trakcie treningu już na pierwszych dwóch epokach. Od tego momentu strata uczenia sieci powoli zbiega do wartości około 2.8×10^{-2}.



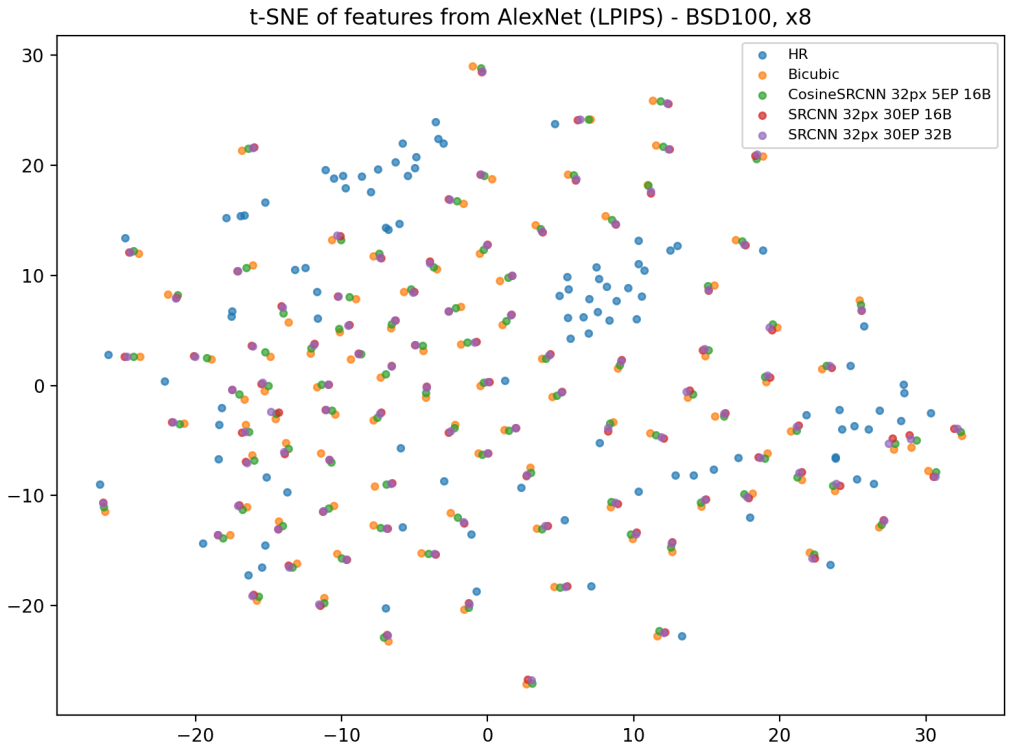
Rysunek 27: Rekonstrukcja obrazu przez sieć CosineSRCNN (x4). W porównaniu z wcześniejszymi wynikami krawędzie obiektów są wyraźniejsze, a rozmycie tła mniej zauważalne (obserwacja na pojedynczym obrazie).

```
Benchmark evaluation on BSD100 dataset results:
SRCNN Upscale | 24.684386 dB | SSIM: 0.698341 | LPIPS: 0.404314
Bicubic Upscale | PSNR: 24.022980 dB | SSIM: 0.660729 | LPIPS: 0.519213
```

Rysunek 28: CosineSRCNN osiągnęła najlepsze metryki spośród wszystkich eksperymentów dla skali 4x: +0.661 dB PSNR, +0.038 SSIM oraz -0.115 LPIPS względem metody bazowej. W porównaniu z SRCNN trenowaną 60 epok przewaga wynosi +0.158 dB PSNR i -0.022 LPIPS, a SSIM rośnie jedynie o 0.008. Przewaga ta okupiona jest ok. 14-krotnie większą liczbą parametrów sieci i dłuższym czasem trwania epoki.



Rysunek 29: Porównanie t-SNE między rekonstrukcją obrazu a interpolacją bikubiczną dla doświadczenia SR x4.



Rysunek 30: Porównanie t-SNE między rekonstrukcją obrazu a interpolacją bikubiczną dla doświadczenia SR x8.

Obserwacje: Na obu ilustracjach rekonstrukcje wszystkich sieci leżą bardzo blisko punktów odpowiadających interpolacji bikubicznej i trudno wzrokowo odróżnić poszczególne modele. Dla skali x8 punkty HR są wyraźnie odsunięte od skupisk rekonstrukcji, natomiast dla x4 przesunięcie wydaje się mniejsze.

6 Wnioski

W ramach przeprowadzonych eksperymentów wszystkie trenowane sieci poprawiają jakość względem interpolacji bikubicznej na zbiorze BSD100. Zastosowanie architektury Cosine SRCNN pozwala na zwiększenie wyników metryk (najlepszy przypadek dla SR x4). Jest to obarczone wysokim nakładem czasowym i obliczeniowym.

W celu uzyskania względnie porównywalnych wyników możliwe jest wykorzystanie prostych siatek konwolucyjnych, pozwalają na przeprowadzenie znacznie szybszego treningu kosztem zaniżonego PSNR na zbiorze testowym.

Metryki ewaluacyjne nie pozwalają w pełni ocenić, czy różnice w rekonstruowanym obrazie są zauważalne dla odbiorcy (zwłaszcza przy oglądaniu z dystansu); rozstrzygnięcie wymagałoby subiektywnej oceny jakości. Wyniki pochodzą z pojedynczych przebiegów treningu i wymagają potwierdzenia na wielu ziarnach losowych (w doświadczeniach wykorzystywane było tylko jedno ziarno dla odtwarzalności).